

Nettoyer un Excel pourri

10 étapes pandas + code



Le code exact, le piège fréquent et le résultat visible, étape par étape, sur un seul fichier fil rouge.

Le fichier fil rouge de ce guide s'appelle **export_ventes_brut.xlsx** : trois feuilles (dont un tableau de synthèse qui ressemble à des données brutes), des en-têtes avec espaces et casse mélangée, des types mixtes dans une même colonne, des dates en deux formats différents et des valeurs manquantes cachées derrière "N/A", "-" ou une case vide. C'est un fichier banal, du genre qu'on reçoit un vendredi soir. Sur le terrain, un nettoyage bâclé sur ce type de fichier ne plante jamais bruyamment : il produit un chiffre faux qui part en reporting sans erreur ni warning. Les 10 étapes qui suivent transforment ce fichier en dataset propre, avec à chaque fois le code, le piège et le résultat avant/après.

Palier 1 : diagnostiquer et cadrer

Avant de nettoyer quoi que ce soit, tu dois savoir ce que tu as vraiment entre les mains : la bonne feuille, les bons noms de colonnes.

1. Charger et inspecter le fichier

Cas : export_ventes_brut.xlsx contient trois feuilles : Recap, Ventes_Q1 et Ventes_Q2. La feuille Recap (un tableau croisé déjà résumé par un collègue) a été laissée en première position. Avant tout nettoyage, confirme quelle feuille contient les données brutes.

```
import pandas as pd

xls = pd.ExcelFile("export_ventes_brut.xlsx")
print(xls.sheet_names)
# ['Recap', 'Ventes_Q1', 'Ventes_Q2']

df = pd.read_excel(xls, sheet_name="Ventes_Q1")
df.info()
df.head()
```

Piège fréquent

pd.read_excel("export_ventes_brut.xlsx") sans sheet_name charge silencieusement la première feuille (index 0). Ici c'est Recap, un tableau déjà agrégé, pas les ventes brutes. Aucune erreur n'est levée : tout le nettoyage démarre sur la mauvaise base sans que tu le voies.

Résultat visible

Le même chargement, deux réalités très différentes.

Sans sheet_name explicite

df.shape donne (14, 5) : un résumé déjà agrégé, pris pour des données brutes.

Avec sheet_name="Ventes_Q1"

df.shape donne (480, 9) : les 480 ventes réelles de la feuille.

2. Nettoyer les noms de colonnes

Cas : Les en-têtes bruts mélangent "Client", "Ville" et "Ville " (espace final laissé par un export précédent), "Prix Unitaire", etc. Une normalisation trop rapide peut faire fusionner deux colonnes différentes en une seule sans prévenir.

```
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_", regex=False)
)

print(df.columns.tolist())
print(df.columns.duplicated().sum())
```

Piège fréquent

"Ville" et "Ville " deviennent toutes les deux `ville` après `strip/lower`. pandas autorise des colonnes en double sans erreur : `df["ville"]` renvoie alors un DataFrame à 2 colonnes au lieu d'une Series, et une opération censée renvoyer une valeur unique casse plus loin dans le script, souvent plusieurs étapes après le renommage.

Résultat visible

Vérifier tout de suite, pas plus tard dans le script.

Avant

9 colonnes brutes, dont 2 variantes de "Ville" repérées à l'œil seulement.

Après

`df.columns.duplicated().sum() == 0` confirmé par code avant de continuer.

À retenir avant de toucher une seule cellule

Le diagnostic (étapes 1 et 2) doit répondre à 3 questions avant tout nettoyage.

Bonne feuille

Confirmée par `sheet_name` explicite, pas par défaut.

Colonnes uniques

Zéro doublon vérifié par `columns.duplicated()`.

Volume réel

`df.shape` correspond aux vraies ventes, pas à un résumé.

Palier 2 : nettoyer la structure

Une fois la bonne feuille chargée et les colonnes nommées sans collision, tu poses le squelette : lignes vides, types, texte homogène. Sans ça, chaque calcul plus loin part faux.

3. Supprimer colonnes et lignes vides

Cas : Une colonne "commentaire" est presque toujours vide, et quelques lignes sont entièrement blanches à cause d'une mise en forme Excel. Un `dropna()` par défaut est beaucoup trop large pour ce cas.

```
df = df.dropna(axis=1, how="all")
df = df.dropna(axis=0, how="all")

df = df.dropna(subset=["date_vente", "montant_total"], thresh=2)
```

Piège fréquent

`df.dropna()` tout seul utilise `how="any"` sur toutes les colonnes : il supprime toute ligne qui a au moins une valeur manquante, même dans un champ facultatif comme "commentaire". Sur ce fichier, ça efface la quasi-totalité des lignes valides pour un champ qui ne devrait jamais bloquer le nettoyage.

Résultat visible

Le choix des arguments change tout, pas juste le nom de la fonction.

dropna() par défaut

df.shape passe de (480, 9) à (71, 9) : la quasi-totalité des ventes disparaît.

dropna(how="all") + subset ciblé

df.shape passe de (480, 9) à (471, 9) : seules les lignes réellement vides partent.

4. Uniformiser les types

Cas : La colonne `prix_unitaire` mélange des floats propres (12.5) et des chaînes avec symbole ("12 €", "8,90 €"). Convertir directement en numérique casse toute la colonne sans prévenir.

```
df["qte"] = pd.to_numeric(df["qte"], errors="coerce")

df["prix_unitaire"] = (
    df["prix_unitaire"]
    .astype(str)
    .str.replace(chr(8364), "", regex=False)
    .str.replace(",", ".", regex=False)
    .str.strip()
)
df["prix_unitaire"] = pd.to_numeric(df["prix_unitaire"], errors="coerce")

print(df["prix_unitaire"].isna().sum())
```

Piège fréquent

Appliquer `pd.to_numeric(df["prix_unitaire"], errors="coerce")` directement, sans retirer le symbole "€" d'abord, transforme silencieusement toute la colonne en NaN : aucune valeur ne matche un format numérique pur. `errors="coerce"` n'affiche aucun avertissement, tu perds la colonne entière si tu ne comptes pas les NaN produits avant et après.

Résultat visible

Compter les NaN produits, avant de faire confiance à la conversion.

to_numeric direct sur "12 €"

`isna().sum()` proche de 480 sur 480 : la quasi-totalité de la colonne devient NaN.

nettoyage string puis to_numeric

`isna().sum() = 3 sur 480` : seules 3 valeurs sont réellement invalides.

5. Nettoyer les chaînes

Cas : La colonne `ville` contient "Paris", "paris", "PARIS " et "Pari" (faute de frappe) pour la même ville. Un `groupby` sur du texte non normalisé compte plusieurs villes là où il n'y en a qu'une.

```
for col in ["ville", "produit", "vendeur"]:
    df[col] = df[col].astype(str).str.strip().str.title()

df["ville"] = df["ville"].replace({"Pari": "Paris"})

print(df["ville"].nunique())
```

Piège fréquent

"paris", "Paris" et "PARIS " sont trois chaînes différentes tant qu'elles ne sont pas normalisées. `df.groupby("ville")["montant_total"].sum()` calcule alors 3 totaux séparés pour une seule vraie ville : le classement des villes par chiffre d'affaires devient faux, sans qu'aucune erreur ne s'affiche.

Résultat visible

Une catégorie propre au sens pandas, pas seulement lisible à l'œil.

Avant nettoyage

`df["ville"].nunique()` = 14 valeurs uniques, dont des doublons de casse.

Après strip/title + correction

`df["ville"].nunique()` = 9 villes réelles, groupby fiable.

Structure propre = base fiable

Après les étapes 3 à 5 : plus de colonnes dupliquées, types numériques propres, texte normalisé. C'est la base indispensable avant de toucher aux dates et aux valeurs manquantes.

Palier 3 : nettoyer les valeurs

Les colonnes existent, les types sont bons. Reste le plus piégeux : les dates, les valeurs manquantes cachées derrière du texte, et les doublons presque invisibles.

6. Convertir les dates

Cas : La colonne `date_vente` mélange deux graphies : "05/03/2024" (jour/mois) et "2024/06/01" (ISO). Laisser pandas deviner le format ligne à ligne mélange silencieusement jour et mois.

```
df["date_vente"] = pd.to_datetime(
    df["date_vente"], format="%d/%m/%Y", errors="coerce"
)

non_converties = df["date_vente"].isna().sum()
print(f"{non_converties} dates non converties avec ce format, à vérifier une par une")
```

Piège fréquent

Appeler `pd.to_datetime(df["date_vente"])` sans argument `format` laisse pandas inférer un format par ligne. "03/04/2024" peut être lu comme le 3 avril pour une ligne et le 4 mars pour une autre, selon l'inférence, sans exception levée. Toujours fixer un format explicite, et isoler avec `errors="coerce"` les lignes qui ne matchent pas plutôt que de laisser une date fausse mais valide passer inaperçue.

Résultat visible

L'inférence ne prévient jamais quand elle se trompe.

to_datetime sans format

Dates incohérentes non détectables : même graphie, deux interprétations selon la ligne.

to_datetime avec format explicite

6 lignes NaT isolées pour vérification manuelle, le reste fiable à 100%.

7. Traiter les valeurs manquantes

Cas : La colonne `statut` contient des "-", des cases vides et parfois "N/A". Les `na_values` par défaut de `read_excel` couvrent déjà "N/A" et les chaînes vides, mais pas "-" ni un placeholder maison.

```
import numpy as np

NA_VALUES_MAISON = ["-", "NC", "nc", "n.c.", "inconnu"]

df = df.replace(NA_VALUES_MAISON, np.nan)

print(df["statut"].isna().sum())
print(df["statut"].value_counts(dropna=False))
```

Piège fréquent (vérifié)

`isna()` ne voit pas les placeholders "N/A maison" non déclarés : "-", "NC" ou un placeholder maison restent du texte tant que rien ne les convertit, et un `groupby` sur `statut` les traite comme une vraie catégorie au lieu d'une valeur manquante. Si tu utilises `na_values` au chargement, déclare-les dès l'étape 1 ; sinon, comme ici sur un `df` déjà en mémoire, remplace-les explicitement avec `df.replace(...)`.

Résultat visible

Déclarer, pas espérer que pandas devine tout seul.

avant remplacement explicite

`df["statut"].isna().sum() = 0`, alors que 38 lignes contiennent "-".

après `df.replace(NA_VALUES_MAISON, np.nan)`

`df["statut"].isna().sum() = 38`, valeurs manquantes vraiment visibles.

Sur le terrain : ces deux pièges (dates mélangées, placeholder "-" non déclaré) sont la cause la plus fréquente d'un chiffre faux qui part en reporting sans qu'aucune erreur pandas ne s'affiche. Un traceback, tu le vois tout de suite. Un NaN silencieux, tu ne le vois jamais.

8. Dédupliquer

Cas : Deux lignes concernant la même vente du même client, à la même date, pour le même montant. Mais l'une porte "Julie Martin" et l'autre "julie martin " avec un espace en trop. Une comparaison brute ne les voit pas comme des doublons.

```
df["client_norm"] = df["client"].astype(str).str.strip().str.lower()

doublons = df.duplicated(
    subset=["client_norm", "date_vente", "montant_total"], keep=False
)
print(df.loc[doublons, ["client", "date_vente", "montant_total"]])

df = df.drop_duplicates(
    subset=["client_norm", "date_vente", "montant_total"], keep="first"
)
```

Piège fréquent

`df.drop_duplicates()` sans `subset` compare les colonnes brutes telles quelles. "Julie Martin" et "julie martin " diffèrent d'un espace et d'une casse : pandas les traite comme deux lignes différentes et garde le doublon. Il faut normaliser une colonne clé avant de chercher les doublons, jamais comparer le texte brut.

Résultat visible

Normaliser avant de comparer, pas après.

`duplicated()` sur colonnes brutes

`df.duplicated().sum() = 0`, alors que 14 doublons quasi identiques existent.

`duplicated()` sur `client_norm + subset`

14 doublons détectés et supprimés, `df.shape` passe de 471 à 457 lignes.

9. Corriger les aberrantes et incohérences métier

Cas : Deux lignes sautent aux yeux : un `montant_total` à -1€ (probable erreur de saisie) et un `montant_total` à 48000€ (peut-être un vrai grand compte). Un filtre appliqué sans trace les traite comme si c'était pareil.

```
masque_aberrant = (df["montant_total"] <= 0) | (df["montant_total"] > 20000)

log_exclusions = df.loc[
    masque_aberrant, ["client", "date_vente", "montant_total"]
].copy()
log_exclusions.to_csv("lignes_exclues_montant.csv", index=False)

df = df.loc[~masque_aberrant].copy()
print(f"{masque_aberrant.sum()} lignes exclues, loggées dans lignes_exclues_montant.csv")
```

Piège fréquent

Écrire `df = df[df["montant_total"] > 0]` sans garder trace des lignes retirées les supprime définitivement. Le montant à -1€ (erreur) et le montant à 48000€ (peut-être une vraie vente exceptionnelle) disparaissent ensemble, sans que personne ne puisse vérifier après coup si un cas métier réel a été perdu dans le nettoyage.

Résultat visible

Une exclusion qui laisse une preuve relisible.

filtre sans log

Le total de lignes baisse de 457 à 442 sans justification traçable.

masque explicite + log_exclusions.csv

15 lignes exclues, listées avec client, date et montant dans un fichier relisible.

Ne jamais filtrer sans traçabilité

Toute suppression de ligne doit laisser une trace : combien de lignes, lesquelles, pour quelle raison. Un masque booléen sans log n'est pas un nettoyage, c'est une perte de données silencieuse.

Palier 4 : finaliser et fiabiliser

Le fichier est propre. Reste à le rendre fiable et réutilisable : contrôles automatiques avant export, et un export versionné plutôt qu'un bricolage à refaire chaque mois.

10. Valider et exporter

Cas : `export_ventes_brut.xlsx` revient chaque mois avec les mêmes pièges. Un nettoyage refait à la main dans un notebook jamais sauvegardé recommence de zéro à chaque fois.

```
assert df["montant_total"].isna().sum() == 0, "montant_total contient encore des NaN"
assert df["date_vente"].isna().sum() == 0, "date_vente contient encore des NaT"
assert df.columns.duplicated().sum() == 0, "colonnes dupliques detectees"
assert not df.duplicated(subset=["client_norm", "date_vente", "montant_total"]).any()

df.to_csv("ventes_propre_2024.csv", index=False)
df.to_parquet("ventes_propre_2024.parquet", index=False)
```

Piège fréquent

Exporter un fichier propre depuis un notebook bricolé, sans script versionné, transforme chaque nouveau `export_ventes_brut.xlsx` du mois suivant en travail entièrement à refaire. Personne, pas même toi dans six mois, ne peut relancer ou auditer exactement les mêmes étapes.

Résultat visible

Un script versionné change le coût du mois prochain.

nettoyage artisanal chaque mois

Envir 2 heures reperdues à chaque nouvel export, aucune cohérence garantie.

script `clean_ventes.py` versionné

Relance en quelques secondes, 4 asserts qui bloquent l'export si une règle est violée.

Du brut au propre : ce que 10 étapes changent

Avant (fichier brut)

480 lignes, feuille de départ incertaine (Recap en position 0).

Colonnes potentiellement dupliquées après renommage naïf.

Statut "-" invisible pour `isna()`.

Dates JJ/MM et ISO mélangées, erreur silencieuse possible.

14 doublons quasi identiques non détectés.

Aucun export reproductible.

Après (10 étapes)

442 lignes propres, chaque suppression tracée.

Colonnes uniques vérifiées : 0 doublon.

38 valeurs manquantes explicitement détectées.

Format explicite, 6 dates ambiguës isolées.

14 doublons supprimés après normalisation.

CSV et Parquet versionnés, avec 4 asserts de qualité.

Verdict terrain : un Excel pourri n'est pas un problème de compétence Excel, c'est un problème de méthode pandas. Le même fichier, avec le même script, prend 5 minutes le mois prochain au lieu de 2 heures. Un workflow scripté n'est pas un nettoyage jetable : c'est un actif réutilisable.

Fait main, en solo.

Si ce guide t'a fait gagner du temps sur ton prochain fichier pourri, partage-le autour de toi : il peut débloquent un autre analyste. Et si tu veux que je traite un autre pipeline pandas précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

